HONEYWELL SOFTWARE ENGINEERING TEAMS — IA · BA · PA&T

AI Champions Programme — GitHub Copilot Enterprise, SDD, MCP & Agentic Software Engineering

Module 04

Spec-Driven Development with GitHub Spec Kit

Why specifications beat one-off prompts, what a good spec contains, how the SDD tool landscape compares — then a deep dive into GitHub Spec Kit's workflow and the best practices that keep specs, plans and code in sync.

DURATION
2 hours

FORMAT
Concepts + Spec Kit lab

PRIMARY TOOL
Copilot + GitHub Spec Kit

How We'll Spend These Two Hours

From why specs matter, to the tools, to a hands-on Spec Kit run that you compare with a prompt-only baseline.

01

Why Spec-Driven Development

15 min

Where prompt-only work drifts, and the spec → plan → tasks → code chain that fixes it.

02

Anatomy of a Good Specification

20 min

Requirements, constraints, contracts and acceptance criteria — weak vs. strong.

03

The SDD Tool Landscape

15 min

Spec Kit, OpenSpec, Kiro, BMAD, Task Master, Tessl — and how to choose.

04

GitHub Spec Kit Deep Dive

35 min

Constitution, specify, clarify, plan, tasks, analyze, implement — with the artifacts each creates.

05

Best Practices & Change Control

10 min

Do/don't list, traceability, and keeping specs alive as code changes.

06

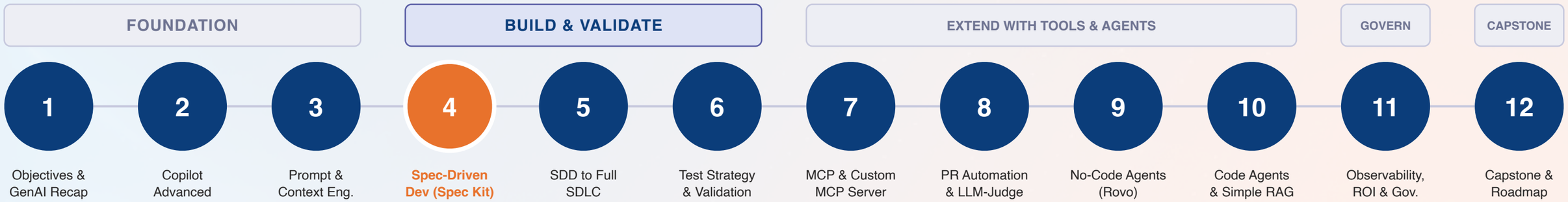
Lab: Spec Kit vs Prompt-Only

25 min

Build one feature both ways; compare quality, rework, tokens and time.

Module 04 in the 12-Module Journey

The first module of the BUILD & VALIDATE phase: specifications become the contract everything else is checked against.



M03

Context engineering: a spec is the densest, most durable context you can write

M05

Takes today's spec through architecture, tests, CI/CD, PR and release

M06

Acceptance criteria in the spec become the test strategy

M08

PR automation and LLM-judge check code against the spec

Why this matters: AI writes code fast. A spec is how you decide in advance what “correct” means — and how reviewers, tests and agents check it afterwards.

Why SDD: Prompt-Only Drifts, Specs Anchor

Same feature, two paths. The difference is where the intent lives.

PROMPT-ONLY

Idea in chat
Intent lives in a long conversation

→

Code appears
Model fills every gap with a guess

→

Review by feel
“Looks right” — no shared definition of done

→

Rework loop
Fix, re-prompt, drift from original intent

SPEC-DRIVEN

Spec
Intent, constraints, acceptance criteria

→

Plan
Architecture and decisions, checked vs spec

→

Tasks
Small, ordered, traceable units of work

→

Code + checks
Each task verified against the spec

- **Intent is explicit**
- **Work is repeatable by humans and agents**
- **Review compares to a contract**
- **Intent survives the chat session**

Core idea: in SDD the specification is the source of truth; code is one output of it. You review the spec before you review the code.

The SDD Chain: Spec → Plan → Tasks → Implementation

Each stage has an artifact, a question it answers, and a human review gate before the next stage.

Specify

WHAT and WHY?

`spec.md`

GATE

Is the intent right?

Plan

HOW, with which tech?

`plan.md`

GATE

Does design fit the spec and standards?

Tasks

In what ORDER?

`tasks.md`

GATE

Are tasks small and testable?

Implement

BUILD and VERIFY

`code + tests`

GATE

Do acceptance criteria pass?

↺ **When code or review reveals a gap, fix the SPEC first, then regenerate downstream — never patch only the code.**

Human in the loop: AI drafts every artifact; engineers own the gates. Reviewing a one-page spec is far cheaper than reviewing 800 lines of code that solve the wrong problem.

Anatomy of a Good Specification

Six building blocks. Each one removes a category of guessing from the model.

Users & scenarios

Who does what, in prioritised user stories

Functional requirements

Testable “MUST” statements

Non-functional constraints

Performance, security, compliance, platform

Interfaces & contracts

APIs, schemas, events, error shapes

Acceptance criteria

Given / When / Then — how we know it is done

Out of scope & open questions

What we deliberately do not build; what is unclear

weak-requirement.md

```
Users should be able to refund orders
quickly and securely.

// vague · untestable · no limits
```

strong-requirement.md

```
FR-3: A support agent MUST be able to refund a
  Paid order up to its original amount.
AC: Given a Paid order of 100,
    When agent refunds 150, Then request is
    rejected with error REFUND_EXCEEDS_TOTAL.
```

Quality test: could a stranger write a test from each line without asking you a question? If not, it is not specific enough. (Examples are illustrative.)

The SDD Tool Landscape

Different tools put the spec in different places. Know what each one is optimised for.

GitHub Spec Kit

TODAY'S FOCUS

Open-source CLI + slash commands. Specify → plan → tasks → implement loop; works with many agents incl. Copilot.

OpenSpec

Change-driven

Change-based: propose → apply → archive. Small, reviewable change units; specs stay aligned with what shipped.

AWS Kiro

IDE-integrated

A spec-driven IDE that enforces requirements → design → tasks inside the editor.

BMAD-METHOD

Agent-team

Multi-agent planning: analyst, PM, architect, scrum-master and dev agents produce PRD and story files.

Task Master AI

PRD → tasks

Parses a PRD into dependency-ordered tasks exposed to agents over MCP. Does not write the spec for you.

Tessl

Spec registry

Specs as persistent agent memory, with a registry of library specs to reduce invented APIs. Framework in closed beta.

Common thread: all six keep intent in versioned text files that humans review and agents read. They differ in workflow shape, agent support and how much process they impose.

Choosing a Tool: Greenfield vs Brownfield

Match the tool to the shape of the work, not to the hype.

WHAT IS THE WORK?

New feature or app from scratch	**Spec Kit** — Full constitution → spec → plan → tasks loop gives structure from day one.
Changing an existing codebase	**OpenSpec** — Change proposals describe deltas against current behaviour and archive afterwards.
Team wants the editor to enforce the process	**Kiro** — Spec mode is built into the IDE; fewer files to wire up.
Large feature, many roles	**BMAD-METHOD** — Role-based agents produce PRD, architecture and stories before code.

Brownfield reality

- Recover the implicit spec first (what does it do today?)
- Make impact analysis and regression criteria explicit
- Iterate — don't expect a perfect upfront spec

Today: we use Spec Kit for the lab. The same habits — explicit intent, review gates, traceability — transfer to any of these tools. Module 05 returns to brownfield work.

GitHub Spec Kit: What You Install and What It Creates

An open-source toolkit: one CLI to scaffold, plus slash commands your coding agent runs.

1 · INITIALISE

terminal

```
# one-time, in your project folder
uvx --from git+https://github.com/
  github/spec-kit.git \
  specify init my-app --ai copilot
```

Check the Spec Kit README for current flags and supported agents before the session.

Works with many agents — Copilot, Claude Code, Cursor, Gemini CLI, Windsurf, Codex CLI and more — with the same structure.

2 · SCAFFOLD

your repo

```
.specify/
  memory/constitution.md
  templates/
specs/001-refund-orders/
  spec.md
  plan.md
  research.md  · data-model.md
  contracts/  · quickstart.md
  tasks.md

// feature folders are numbered
```

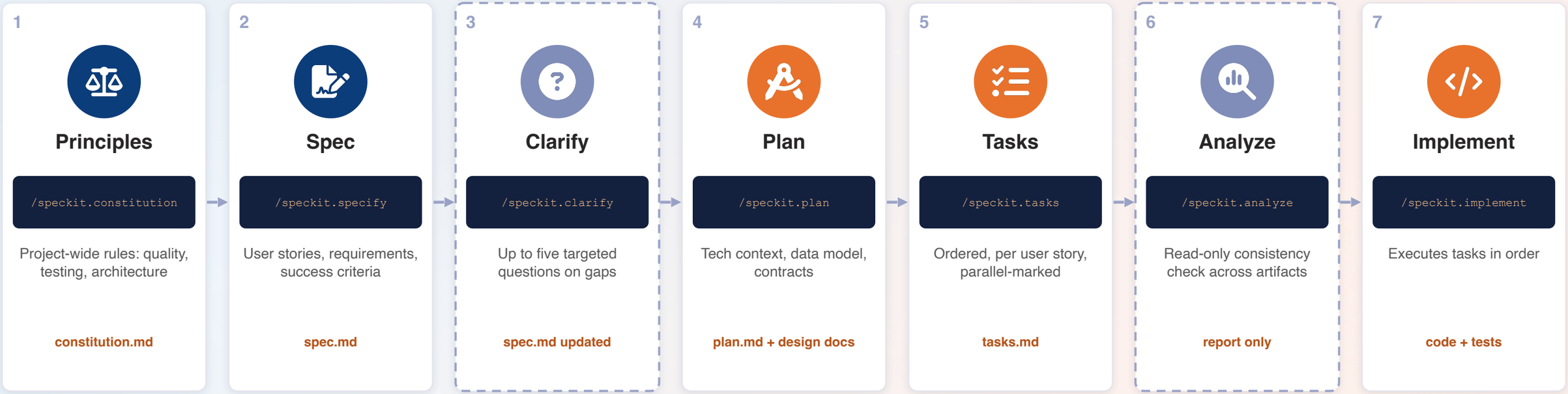
3 · RUN COMMANDS IN CHAT

```
/speckit.constitution
/speckit.specify
/speckit.clarify
/speckit.plan
/speckit.tasks
/speckit.analyze
/speckit.implement
```

Key idea: the commands are just structured prompts + templates. Everything they produce is plain Markdown in git — reviewable in a PR like any other code.

The Spec Kit Workflow, End to End

Seven commands, left to right. Dashed steps are optional but strongly recommended.



Rhythm: run one command, read the artifact, edit it by hand if needed, then move on. The review between commands is the point — not a speed bump.

Steps 1–2: Constitution and Specify

Set the rules once; then describe each feature as what and why — not how.

STEP 1 · /speckit.constitution — PROJECT-WIDE, WRITTEN ONCE

memory/constitution.md (illustrative)

```
## Principles
1. Test-first: no task is done without a
   passing test for its acceptance criteria
2. Layered: controllers → services → repos
3. Public APIs are versioned and documented
4. No new dependency without a recorded reason

## Governance
Amendments need a PR and a reviewer.
```

STEP 2 · /speckit.specify — PER FEATURE

specs/001-refund-orders/spec.md (illustrative)

```
## User story 1 (P1)
As a support agent, I can refund a paid order
so that customers are made whole quickly.

## Requirements
FR-1 Refund <= original amount
FR-2 Only orders in status Paid
[NEEDS CLARIFICATION: partial refunds?]
```



Constitution = guardrails

Every later plan is checked against it.



Spec = what and why

Leave tech stack out; that is the plan's job.



Mark unknowns

Flag open questions instead of letting the model guess.

Best practice: describe the outcome and the user, not the implementation. If your spec names a framework, you have started the plan too early.

Steps 4–5: Plan and Tasks

The plan decides how; the task list makes it executable, ordered and traceable.

/speckit.plan → DESIGN ARTIFACTS

```
specs/001-refund-orders/

plan.md        tech context + decisions
research.md    options considered
data-model.md  entities, fields
contracts/       API / event schemas
quickstart.md  how to verify
```

Plan includes a Constitution Check: does the design break any principle? Justify or fix before moving on.

/speckit.tasks → tasks.md

```
specs/001-refund-orders/tasks.md  (illustrative)

## Phase 1 · Setup
- [ ] T001 Create RefundService skeleton in src/Orders/
## Phase 2 · User Story 1 (P1)
- [ ] T002 [P] [US1] Test: refund over total rejected (FR-1)
- [ ] T003 [P] [US1] Test: unpaid order rejected (FR-2)
- [ ] T004 [US1] Implement Refund() in RefundService.cs
- [ ] T005 [US1] Add POST /orders/{id}/refund endpoint

// [P] = can run in parallel · IDs are stable · paths are exact
```

Stable IDs

T001… cited in commits and PRs

[P] = parallel

Independent tasks can run side by side

Story labels [US1]

Every task traces to a requirement

Best practice: tell the plan your tech choices and constraints explicitly (stack, existing services, limits). If you don't, the model picks for you — review the plan like an architecture proposal.

Quality Gates: Clarify, Checklist, Analyze

Three cheap checks that catch problems before any code exists.

/speckit.clarify

After specify, before plan

Asks up to five targeted questions about gaps and ambiguity, then records your answers in a Clarifications section of the spec.

YOU GET

Fewer silent assumptions

/speckit.checklist

Anytime after specify

Generates checklists that test the requirements themselves: complete, clear, consistent, measurable.

YOU GET

Requirements quality, not code quality

/speckit.analyze

After tasks, before implement

Read-only scan across spec, plan and tasks for gaps, inconsistencies and constitution violations. Changes no files.

YOU GET

A report to act on

Best practice: a wrong line in a spec costs minutes to fix; the same gap found in code costs a review cycle and a rewrite. Treat these gates as mandatory, not optional.

Step 7: Implement with Copilot — and Keep It on Rails

Spec Kit supplies the plan; Copilot agent mode does the typing; you hold the gates.

Open repo in VS Code

Spec Kit commands are available in Copilot Chat

Agent mode

Run /speckit.implement; it works through tasks.md in order

Tests first

Each task's tests must pass before it is ticked

Review the diff

Compare against spec.md and acceptance criteria

Commit per task

Message cites the task ID, e.g. T004

KEEP THE AGENT ON RAILS

- Implement one phase or user story at a time, not the whole list
- If the model wants to deviate, change the spec or plan first
- Start a fresh chat per phase; tasks.md carries the state (Module 03: SCOPE)
- Stop and re-run /speckit.analyze if artifacts drift

Take it to your team

/speckit.taskstoissues exports tasks.md as GitHub Issues, so work can be assigned, tracked and linked to PRs across engineers and agents.

Best practice: never accept a task as done on the model's say-so. Run the tests, read the diff, and tick the box yourself.

Spec Kit Best Practices

Habits that separate teams who get leverage from teams who generate paperwork.

DO THIS	AVOID THIS
Write the constitution first and keep it short and enforceable	Paste a vague prompt into /speckit.specify and accept the first draft
Describe outcomes and acceptance criteria; keep the stack for the plan	Put framework or library choices into the spec
Run clarify and analyze every time — they are the cheapest checks	Skip straight to implement because “it looks fine”
Review and edit each artifact by hand before running the next command	Run the whole task list in one long chat
Keep spec, plan and tasks in git; review them in PRs	Patch code to fix a gap and leave the spec stale
Fix the spec first, then regenerate downstream artifacts	Over-spec a one-line fix — match ceremony to risk

Calibrate: a 10-line bug fix does not need a constitution amendment and four documents. Use the full loop for features, the light path for small changes.

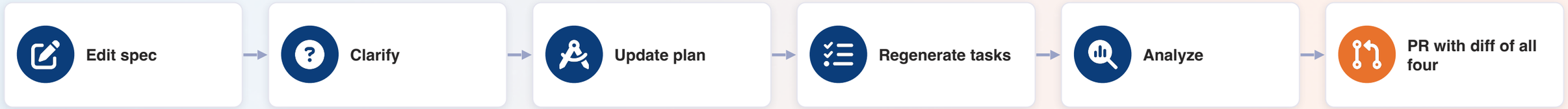
Traceability and Change Control

Specs only stay valuable if they change with the code. Build the trail, then keep it alive.

TRACE A REQUIREMENT FORWARD



WHEN REQUIREMENTS CHANGE



Best practice: make “spec changed?” a PR-template question and a review check. In Module 08 an LLM-judge can enforce it automatically against the spec.

Illustrative IDs shown; your repo's numbering will differ.

Lab: Spec Kit vs Prompt-Only on the Same Feature

LAB 1 · 25 min

Build one small feature twice and measure the difference — not just feel it.

Goal Implement the same feature with a prompt-only chat (baseline) and with the Spec Kit loop; compare quality, rework, tokens and time.

STEPS

1. A — baseline: new chat, one prompt describing the feature. Let Copilot build it. Note time, iterations and tests passing.
2. B1 — init Spec Kit; run /speckit.constitution (3–4 principles) then /speckit.specify for the same feature.
3. B2 — run /speckit.clarify, answer the questions, then /speckit.plan and /speckit.tasks; read each artifact before moving on.
4. B3 — /speckit.analyze, then implement. Fill the comparison table: tokens, iterations, time, rework, acceptance criteria met.

FILES YOU CREATE

```
repo/
lab04/
  .specify/memory/constitution.md
  specs/001-feature/spec.md
  specs/001-feature/plan.md
  specs/001-feature/tasks.md
  comparison.md
```

DONE WHEN

comparison.md has both runs side by side, and you can name which spec line prevented a rework you saw in the baseline.

What You Can Do Now

Six outcomes from today — and the habit that ties them together: review the spec before the code.

- ✓ The spec → plan → tasks → implementation chain, with its review gates, is explained
- ✓ The Spec Kit commands, artifacts and best practices are applied end to end
- ✓ A specification with testable requirements and acceptance criteria can be written
- ✓ Requirements are traced to tasks, tests and PRs; changes start in the spec
- ✓ SDD tools (Spec Kit, OpenSpec, Kiro, BMAD, Task Master, Tessl) are matched to the work
- ✓ A spec-driven run has been compared against a prompt-only baseline

NEXT: MODULE 05 — SDD ACROSS THE FULL SOFTWARE DEVELOPMENT LIFECYCLE · 3.5 HOURS

Take a spec through architecture, tests, CI/CD, PR and release readiness — and apply SDD to brownfield code.

THE SPEC KIT LOOP

Constitution → Specify → Clarify → Plan → Tasks → Analyze → Implement

Questions & Discussion

Module 04 — Spec-Driven Development with GitHub Spec Kit